Decomposition of Graphs: Directed Acyclic Graphs

Daniel Kane

Department of Computer Science and Engineering
University of California, San Diego

Graph Algorithms
Data Structures and Algorithms

Learning Objectives

- Understand the difference between directed and undirected graphs.
- Prove that graphs with cycles cannot be linearly ordered.

Outline

1. Motivation
2. Event Ordering
3. DAGs

Directed Graphs

Sometimes we want the edges of a graph to have a direction.

Definition

A directed graph is a graph where each edge has a start vertex and an end vertex.

Examples

Directed graphs might be used to represent:

- Streets with one-way roads.
- Links between webpages.
- Followers on social network.
- Dependencies between tasks.

Directed DFS

Can still run DFS in directed graphs.

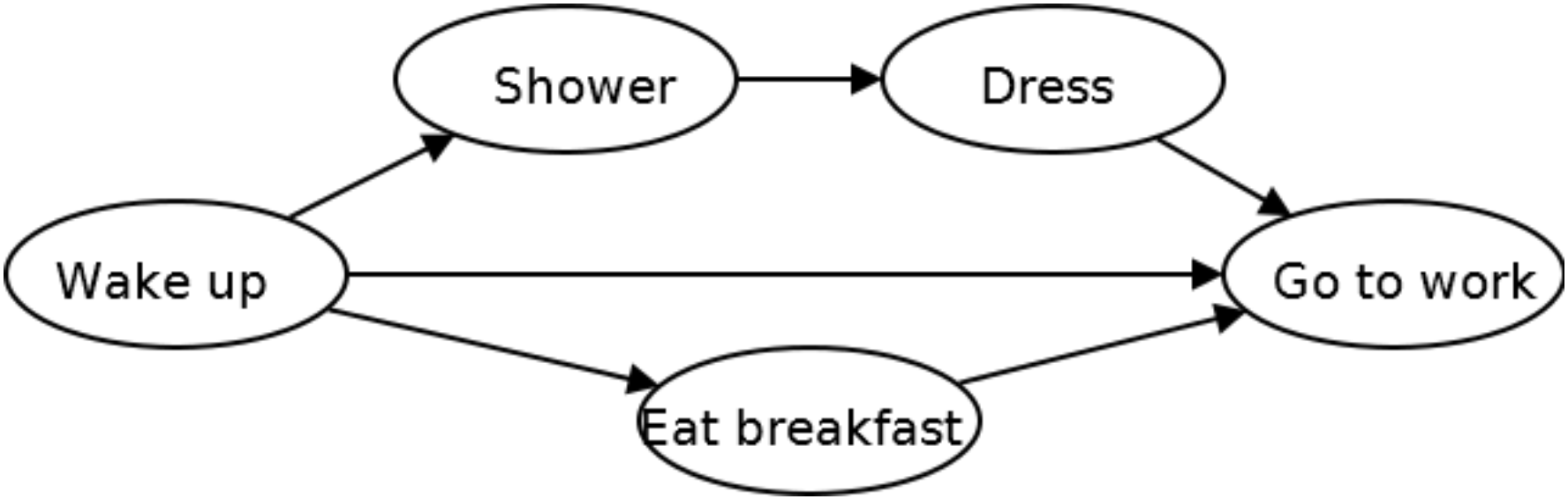
- Only follow directed edges.
- `explore`(v) finds all vertices reachable from v.
- Can still compute pre- and post-orderings.

Outline

1. Motivation
2. Event Ordering
3. DAGs

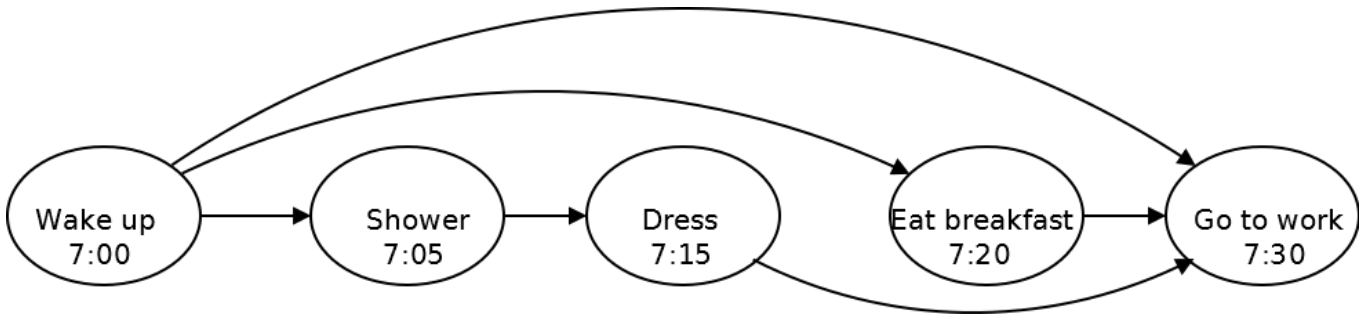
Morning Routine

The following morning tasks must be performed some before others.



Linear Ordering

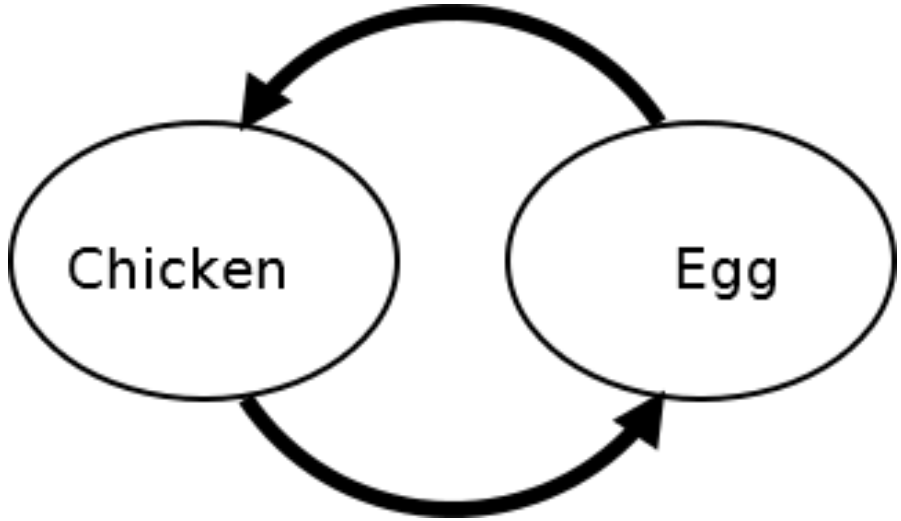
We would like to order tasks to respect dependencies as below.



Is it always possible to do this?

Example

No!



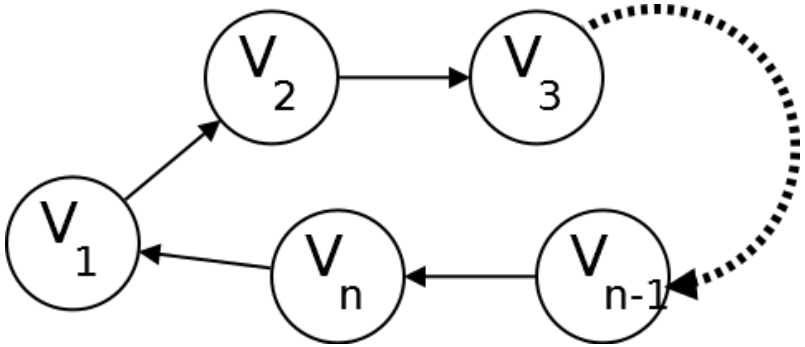
Outline

1. Motivation
2. Event Ordering
3. DAGs

Cycles

Definition

A cycle in a graph G is a sequence of vertices $v_1, v_2, \ldots, v_n$ so that $(v_1, v_2), (v_2, v_3), \ldots, (v_{n-1}, v_n), (v_n, v_1)$ are all edges.



Cycles

Theorem

If G contains a cycle, it cannot be linearly ordered.

Proof

Proof.

- Has cycle $v_1, \ldots, v_n$.
- Suppose linearly ordered.
- Suppose v_k comes first.
- Then v_k comes before v_{k-1}, contradiction.



DAGs

Definition

A directed graph G is a Directed Acyclic Graph (or DAG) if it has no cycles.

DAGs

Definition

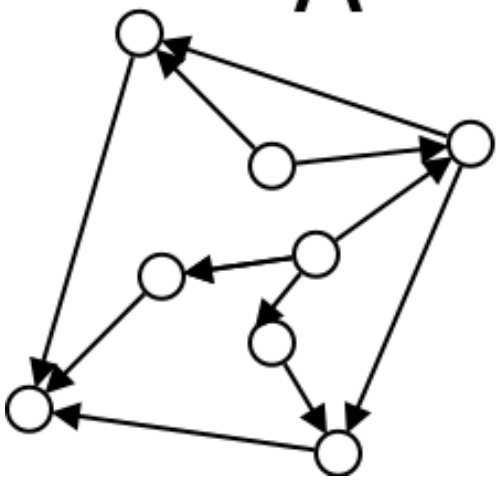
A directed graph G is a Directed Acyclic Graph (or DAG) if it has no cycles.

By the above being a DAG is necessary to linearly order. Is it sufficient?

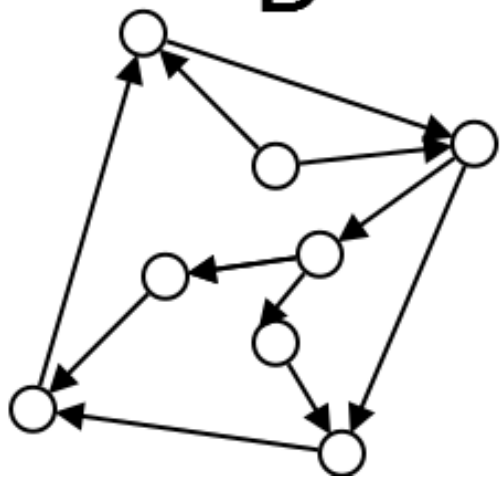
Problem

Which of the following graphs is a DAG?

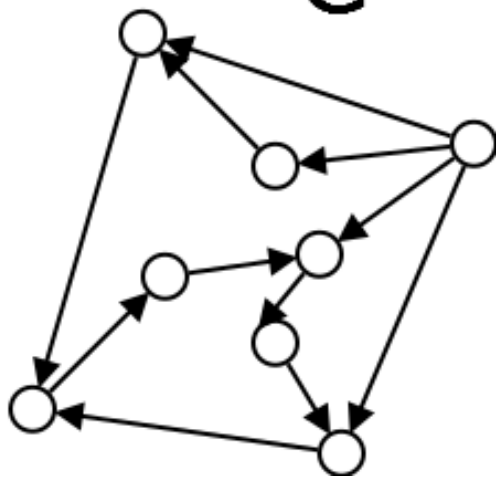
A



B

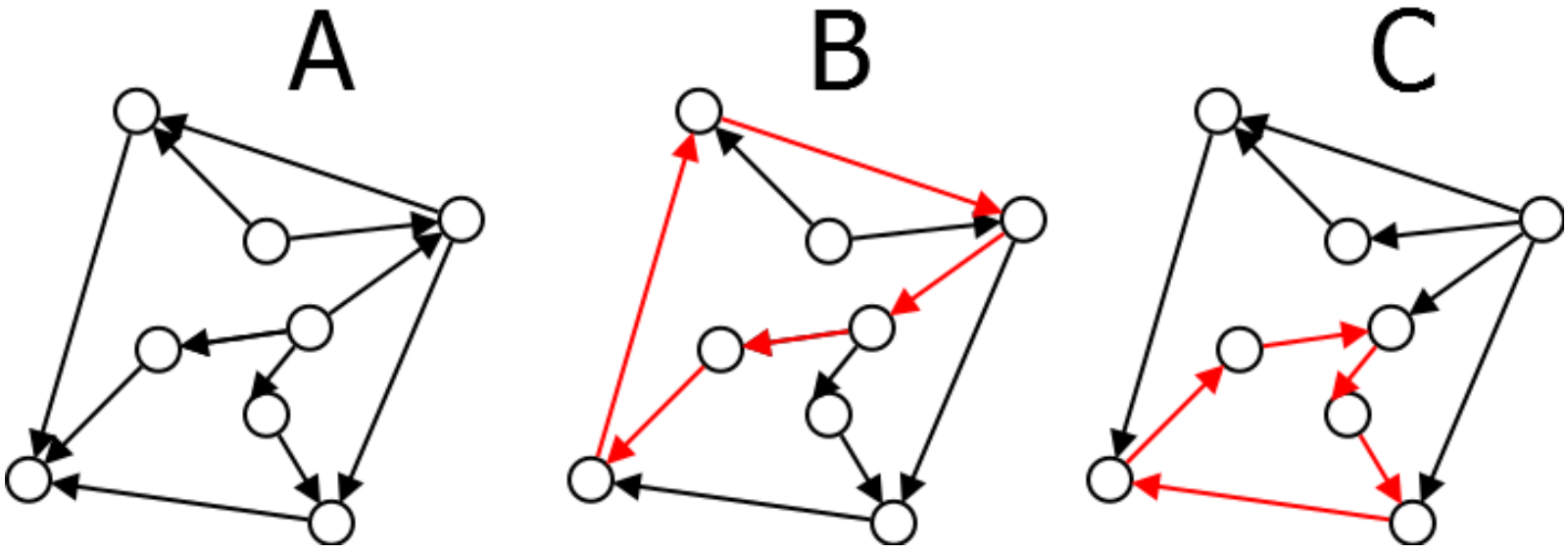


C



Solution

A.



Theorem

Theorem

Any DAG can be linearly ordered.

Next Time

- Prove Theorem
- Develop algorithm